

Informe comparativo

Perceptrón multicapa implementado a mano (NumPy) y en Keras · 4×3×3 vs 4×3×3×3×3

Resumen : Se entrenaron cuatro redes sobre Iris (150 ejemplos, 3 clases): la topología original 4×3×3 y una versión con dos capas ocultas extra (4×3×3×3×3), cada una en NumPy y en Keras. Sigmoide en todas las capas, error cuadrático medio, SGD con $\eta = 0.03$ y 500 épocas. En ambas implementaciones la red más profunda aprendió peor: en NumPy el error pasó de 0.060 a 0.406 y la exactitud de 96.7 % a 70.7 %; en Keras la loss pasó de 0.151 a 0.222 y la exactitud de 66.7 % a 50 %.

1. Resultados de las cuatro corridas

Corrida	Topología	Error / loss final	Exactitud	Tiempo
NumPy A	4×3×3	0.0596	96.7 %	3.2 s
NumPy B	4×3×3×3×3	0.4064	70.7 %	5.9 s
Keras A	4×3×3	0.1509	66.7 %	31.6 s
Keras B	4×3×3×3×3	0.2217	50.0 %	31.4 s

El «error» de NumPy suma las 3 salidas; el MSE de Keras las promedia (error NumPy ÷ 3 = loss Keras).

Semilla 42; los valores pueden variar ligeramente en otra ejecución.

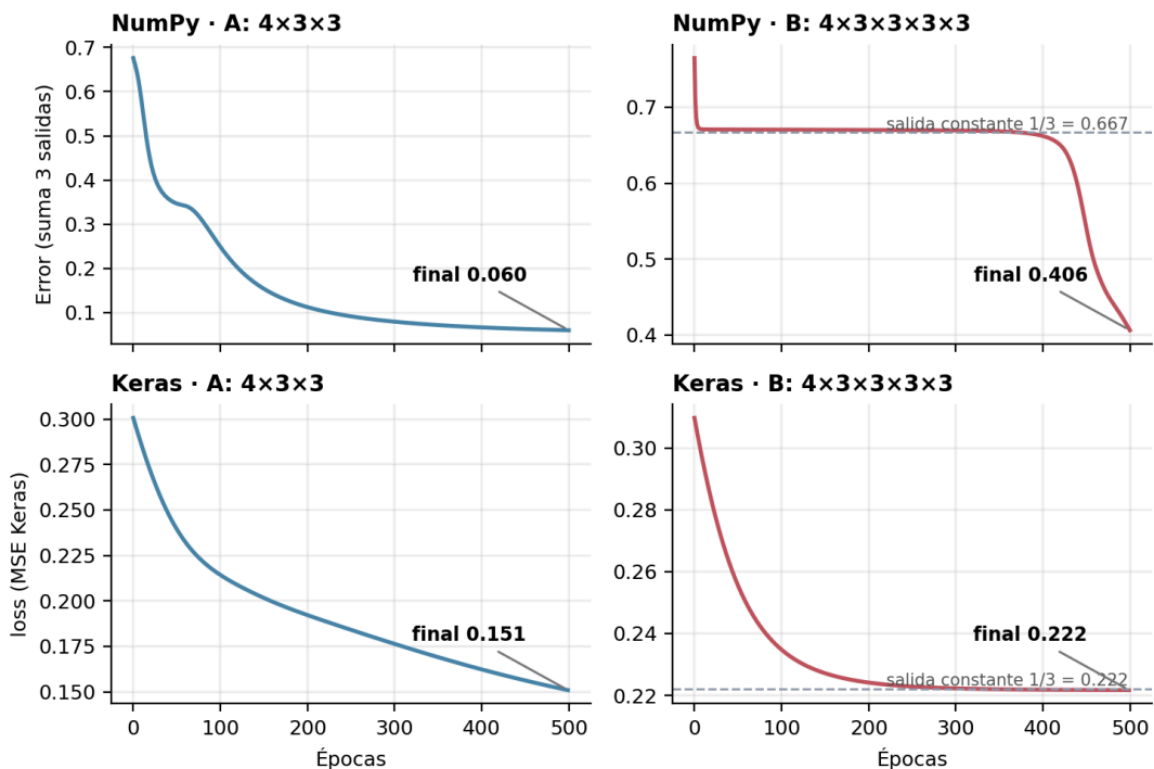


Figura 1. Curvas de error/pérdida de las cuatro corridas.

Las líneas punteadas marcan el error de una salida constante igual a $1/3$ ($2/3$ en NumPy, $2/9$ en Keras): la red profunda permanece en esa meseta: Keras B hasta el final y NumPy B unas 400 épocas, antes de empezar a salir de ella.

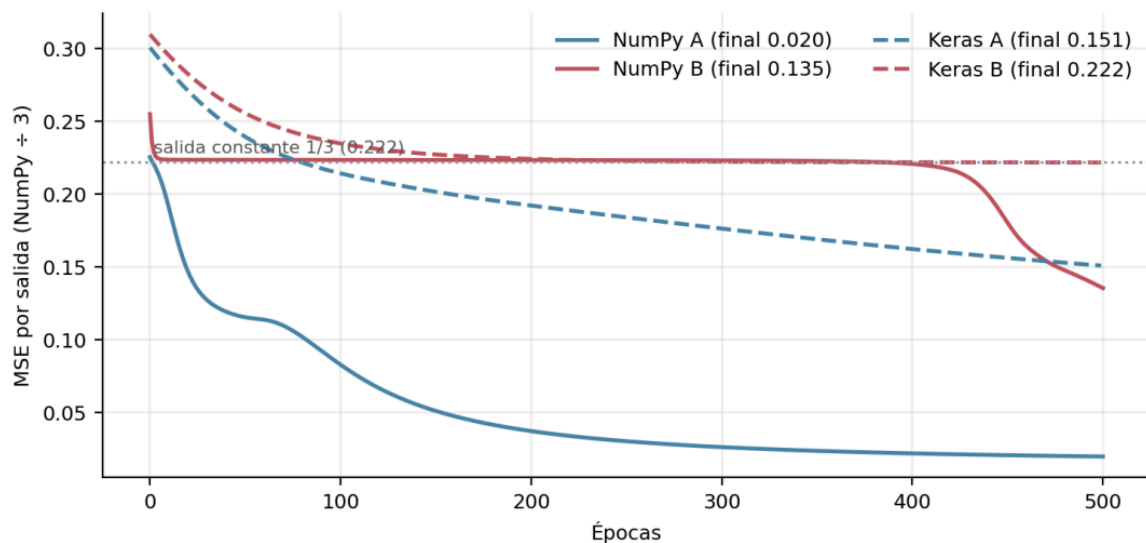


Figura 2. Las cuatro curvas en la misma escala (NumPy dividido entre 3).

2. Análisis

¿Bajó más el error al añadir dos capas? ¿Igual en NumPy y Keras?

No: empeoró en ambas. En NumPy el error final se multiplicó por 6.8 ($0.060 \rightarrow 0.406$) y la exactitud cayó 26 puntos. En Keras la loss subió un 47 % ($0.151 \rightarrow 0.222$) y la exactitud bajó a 50 %. La diferencia está en el grado: la red NumPy profunda aún aprende algo (70.7 %), mientras que la de Keras permanece unas 400 épocas en la meseta de salida constante antes de mejorar y la de Keras termina con una loss de $0.222 \approx 2/9$, es decir, emite $\approx 1/3$ en las tres salidas ($\text{predict de } [3,3,1,1] = [0.342, 0.339, 0.336]$) y no discrimina clases. Además, el entrenamiento fue unas 2 veces más lento en NumPy B (5.9 s vs 3.2 s) por tener más neuronas que recorrer en cada ejemplo.

¿Se parecen las curvas de NumPy y Keras con la misma topología?

No. Con la misma topología $4 \times 3 \times 3$, NumPy llega a 0.0199 por salida ($0.0596 \div 3$) y Keras a 0.151, unas 7.6 veces más. Las causas son de implementación, no de la arquitectura:

- Número de actualizaciones: el código NumPy actualiza los pesos tras cada ejemplo (150 por época, 75 000 en total); Keras usa mini-lotes de 32 (5 por época, 2 500 en total). Con $\eta = 0.03$ Keras avanza mucho más despacio.
- Orden de los datos: NumPy recorre Iris ordenado por clase; Keras baraja en cada época.
- Escala del error: suma de 3 salidas (NumPy) frente a promedio (Keras), factor 3.
- Inicialización: uniforme en $[-0.5, 0.5]$ (NumPy) frente a Glorot con sesgos en 0 (Keras). La vectorización de TensorFlow acelera el cálculo pero no cambia las matemáticas.

Con sigmoides apiladas y MSE, ¿tiene sentido que la red más profunda no aprenda mejor?

Sí. La derivada de la sigmoide vale como máximo 0.25, de modo que cada capa que atraviesa la retropropagación reduce el gradiente. La Figura 4 lo mide en la red NumPy B: en la primera iteración la norma de $\bar{O}$ es 0.24 en la salida y $2.9 \cdot 10^{-4}$ en la primera capa, unas 830 veces menor. Las capas

cercanas a la entrada casi no se actualizan, y la red permanece en la solución trivial «todo 1/3» (Figura 1): NumPy B tarda unas 400 épocas en abandonarla y Keras B no lo logra en 500. Iris es además un problema pequeño y casi separable linealmente, por lo que la capacidad extra no aporta nada.

3. Conclusión y límites

Añadir capas no mejora un problema simple y, con activaciones sigmoide y MSE, lo empeora. El resultado depende de la semilla: en pruebas adicionales con la misma implementación NumPy y cinco semillas, la red profunda terminó con errores entre 0.09 y 0.67, es decir, a veces aprende y a veces queda atascada. Para profundizar, se podría repetir la red profunda con ReLU en las capas ocultas y softmax con entropía cruzada, cambiando una sola variable a la vez.